

MakeAnything

Agent Team Architecture v2

에이전트 팀 설계도

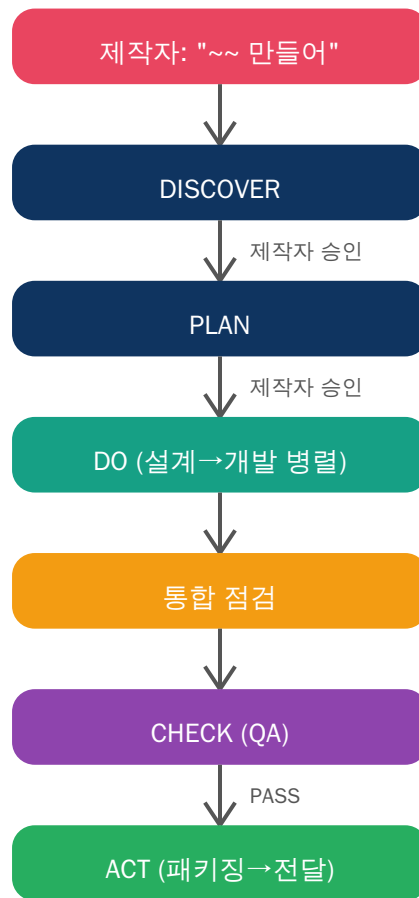
제작자: 정찬 (fokorea5)

날짜: 2026-02-17

에이전트: 코어 5명 + On-Demand 6명 = 11명

흐름: DISCOVER → PLAN → DO → 통합 점검 → CHECK → ACT

1. 전체 흐름 (DPDCA Cycle)



각 단계마다 제작자에게 보고하고 승인을 받은 후 다음으로 넘어갑니다.

가동 조건: "만들어", "개발해", "구현해", "앱", "서비스" 등 제작 의도 감지 시 자동 가동

2. 에이전트 목록 (11명)

▶ 코어 5명 (항상 참여)

에이전트	모델	핵심 역할
오케스트레이터	메인	Team Lead. 코드 안 씀. 흐름 관리, 통합 점검
설계자	opus	DESIGN.md + 통합 계약 작성. src/shared/ 타입
백엔드	sonnet	src/api/, src/db/ 개발. 통합 계약 준수
프론트엔드	sonnet	src/ui/, src/pages/ 개발. 통합 계약 준수
QA	opus	공격자 마인드셋. AC + 통합 계약 검증

▶ On-Demand 6명 (필요 시 소환)

에이전트	모델	소환 조건
보안	opus	auth/pay/security 포함 시 필수
디버거	sonnet	SOS 발생 시 (같은 예러 3회 반복)
DevOps	haiku	1단계: DO 중 도커/CI (투기적) / 2단계: ACT 패키징 (필수)
오라클	sonnet	DISCOVER에서 기술 조사, 외부 API 확인
문서	haiku	투기적 실행. README, docs/ 작성
학습자	haiku	프로젝트 완료 후 교훈 추출 → lessons.db

3. 세포 분화 (프로젝트 규모별 팀 편성)

유형	팀 구성	인원
간단 스크립트	백엔드 + QA	2명
API/백엔드만	설계자 + 백엔드 + QA	3명
풀스택	설계자 + 백엔드 + 프론트 + QA	4명
+인증/결제/보안	위 + 보안 에이전트	+1명
+패키징	위 + DevOps 2단계	+1명

4. 통합 계약 시스템 (Integration Contract)

문제: 설계서에는 dashboard.html인데 프론트가 index.html로 만드는 파일명 불일치

해결: 4중 체크포인트로 불일치 완전 방지

▶ 4중 체크포인트

1. 설계자 → DESIGN.md "통합 계약" 섹션에 정의 (파일명, URL, WS, 환경변수)
2. 개발자 → 통합 계약에 명시된 이름 그대로 사용. 임의 변경 금지
3. 오케스트레이터 → DO→CHECK 사이에 직접 파일 대조 (통합 점검)
4. QA → 통합 계약 vs 실제 코드/파일 불일치 = FAIL

▶ 통합 계약 포함 항목

파일 이름과 경로: 백엔드가 참조하는 프론트 파일의 정확한 이름/경로

URL 라우팅: 프론트가 호출하는 API 경로 = 백엔드가 등록한 라우트

WebSocket/이벤트: WS 경로, 이벤트 이름, 메시지 포맷

환경 변수: 여러 컴포넌트가 공유하는 env 변수의 정확한 키 이름

▶ 통합 점검 체크리스트 (오케스트레이터)

1. DESIGN.md 통합 계약에 명시된 파일이 모두 실제 존재하는가?
2. 백엔드가 참조하는 프론트 파일 이름 = 프론트가 실제 생성한 파일 이름?
3. 프론트가 호출하는 API 경로 = 백엔드가 등록한 라우트?
4. 리다이렉트 경로의 대상 파일이 존재하는가?
5. WebSocket 경로/이벤트 이름이 양쪽에서 동일한가?

5. DISCOVER 단계 상세

▶ 필수 질문 체크리스트 (6개)

1. 어디서 쓸 건가요? — 컴퓨터/핸드폰/웹 + 모바일이면 네이티브/PWA/웹
2. 인터넷 연결 필요한가요? — 서버 필요 여부 판단
3. 서버는 어떻게? — 로컬 vs 배포, 어디에 (서버 필요한 경우만)
4. 핵심 기능은? — 1~3개
5. 만드는 방식 선호? — 언어/프레임워크, "알아서 해주세요"도 OK
6. 완성물 어떻게 받으실래요? — 더블클릭/zip/주소접속/소스코드

▶ PM 마인드셋 (의도 기반 가치 확장)

1. 핵심 의도 추론 — "왜 이걸 만들려는 건가?"를 분석
2. 킬러 기능 도출 — 목적을 더 잘 달성할 기능 1~2개 도출
3. 주도적 역제안 — "이 기능 추가하면 더 좋을 것 같은데, 넣을까요?"

안전장치: "원래대로 해" 시 즉시 수용. 과도한 제안 금지. 핵심 1~2개만.

▶ 전문 용어 번역표 (12개)

용어	쉬운 설명
API	서버와 주고받는 통신 규칙
DB	데이터를 저장하고 꺼내는 저장소
배포	인터넷에 올려서 다른 사람도 쓸 수 있게 하는 것
도커	어떤 컴퓨터에서든 똑같이 실행되게 포장하는 기술
PWA	앱처럼 보이는 웹사이트, 홈 화면 추가 시 앱처럼 동작
네이티브 앱	앱스토어에서 설치하는 앱
WebSocket	실시간 데이터 주고받기 (채팅 등)
CI/CD	코드 변경 시 자동 테스트 + 배포 시스템
서버	요청을 받아서 처리해주는 컴퓨터/프로그램
클라이언트	사용자가 직접 보고 쓰는 화면 쪽
프레임워크	개발을 빠르게 해주는 뼈대 도구
하이브리드 앱	웹 기술로 만들지만 앱스토어에도 올릴 수 있는 앱

6. 에이전트별 상세 규칙

▶ 오케스트레이터 (Team Lead)

코드를 직접 작성하지 않음. DISCOVER→PLAN→DO→통합점검→CHECK→ACT 흐름 관리

PLAN: .plan.md 작성 + 교훈 DB 검색 + Pre-mortem 3개 시나리오

TaskCreate 시: prompts/agents/{에이전트}.md를 읽어서 포함

동시 Task 4개 제한. 공유 타입은 설계자에게 지시

모델 배치: 설계자/QA/보안=opus, 백엔드/프론트/디버거/오라클=sonnet, DevOps/문서/학습자=haiku

▶ 설계자 (Architect)

허용: DESIGN.md, src/shared/ 읽기/쓰기. 금지: 코드 파일 작성

DESIGN.md에 API 엔드포인트, DB 스키마, 디렉토리 구조, 공유 타입 포함

통합 계약 섹션 필수 작성. oracle_report.md 반영

▶ 백엔드 개발자

허용: src/api/, src/db/, src/models/. 금지: .plan.md, DESIGN.md, src/shared/ 수정

DESIGN.md 먼저 읽고 설계 따르기. 통합 계약 이름 그대로 사용

위험 코드에 @risk 태그, 불확실 코드에 @confidence: low 태그

▶ 프론트엔드 개발자

허용: src/ui/, src/pages/, src/styles/. 금지: src/api/, src/shared/ 수정

로딩 상태와 에러 상태 UI 빠뜨리지 않기

통합 계약 파일명/경로 임의 변경 시 백엔드와 연결 끊어짐 주의

▶ QA

공격자 마인드셋으로 검증. AC 기반 테스트 + 통합 계약 검증

통합 계약 vs 실제 파일 불일치 = FAIL

▶ 보안 에이전트

허용: src/ 읽기, 보안 스캔. 금지: 파일 수정

체크리스트: 인증(토큰), 입력(SQL/XSS), 저장(해싱/암호화), 노출(스택트레이스)

설계 변경 필요 시 즉시 FREEZE

▶ 디버거

SOS 발생 시 소환. 에러 분석 → 근본 원인 특정 → 수정 → 테스트

2개 이상 파일 수정 시 범위 보고 후 진행. 환경 문제는 즉시 보고

▶ DevOps (2단계 분리)

1단계 (DO 중, 투기적): CI/CD, Dockerfile 작성. QA FAIL 시 폐기 가능

2단계 (ACT 중, 필수): 실행 스크립트 + zip + 빌드. 건너뛰기 금지

패키징 후 직접 실행 테스트 수행

▶ 오라클

DISCOVER에서 기술 조사. 외부 API 실현 가능성, 제약사항 확인

▶ 문서 에이전트

투기적 실행. README.md, docs/ 작성. QA FAIL 시 폐기

▶ 학습자

프로젝트 완료 후 교훈 추출 → lessons.db에 기록. 중복 병합

7. 공통 규칙 및 안전장치

▶ 파일 소유권

각 에이전트는 자기 소유 파일에만 쓰기. 남의 파일은 읽기만.

공유 타입/인터페이스는 src/shared/에 있으며 설계자만 수정.

▶ 보고 방식

할당 작업 전부 완료 후 1회 보고. 상세는 reports/{자기이름}_report.md

▶ 예외 (즉시 보고)

보안 취약점, 데이터 손실 위험

같은 에러 2회 재시도 후 미해결

.plan.md와 현실 불일치

FREEZE 발견 (코드 수정만으로 해결 불가, 설계 변경 필요)

▶ FREEZE 프로토콜

"FREEZE: [이유]"를 받으면 현재 작업 마무리 후 중지

오케스트레이터의 "작업 재개" 전까지 새 작업 시작 금지

8. ACT 단계 — 패키징 및 전달

▶ 패키징 유형

유형	내용	결과물
바로 실행 (웹앱+서버)	의존성 설치 + 서버 시작	start.bat + start.sh
바로 실행 (클라이언트)	index.html 더블클릭	HTML 파일
zip 패키징	불필요 파일 제외 + 실행방법 포함	project.zip
도커	docker compose up 한 줄 실행	Dockerfile + compose
데스크톱	PyInstaller/electron-builder	exe/app 파일

▶ zip 패키징 규칙

제외 대상: node_modules/, __pycache__/, .git/, .env, *.pyc, venv/
zip 안에 README_실행방법.txt 포함 (3줄 이내, 비개발자도 이해 가능)

▶ 실행 스크립트 규칙

비개발자가 더블클릭만으로 실행 가능해야 함
에러 시 한글로 안내 메시지 출력 (예: "Python이 설치되어 있지 않습니다")

9. 파일 구조

```
makeanything/  
  CLAUDE.md          — 메인 규칙 (가동 조건, 통합 계약, 파일 소유권)  
  prompts/  
    agents/  
      orchestrator.md — DISCOVER→PLAN→DO→통합점검→CHECK→ACT  
      architect.md    — 설계 + 통합 계약 정의  
      developer-backend.md — 백엔드 개발  
      developer-frontend.md — 프론트엔드 개발  
      qa.md           — QA 검증  
      devops.md       — DevOps (2단계)  
      security.md     — 보안 검증  
      debugger.md     — 디버깅  
      oracle.md       — 기술 조사  
      documenter.md   — 문서화  
      learner.md      — 교훈 추출  
  playbook/  
    discover.md       — 질문 체크리스트 + PM 마인드셋 + 용어표
```


qa_guide.md	— QA 상세 가이드
freeze.md	— FREEZE 대응
debugger.md	— 디버깅 플레이북
speculative.md	— 투기적 실행
change_request.md	— 변경 요청
memory/	
lessons_db.py	— 교훈 DB